

Impenetrable Edge Vault: Silicon-Backed SRAM PUF & Fuzzy Extractor

TRON Programming Contest 2026 — RTOS Application Division (Student Category)

Project Metadata

Target Hardware: NUCLEO-H533RE (Cortex-M33) **Primary Language:** C (C11) / Assembly / Python 3

Executive Summary

The **Impenetrable Edge Vault** is a zero-key-storage hardware security solution designed for resource-constrained edge devices. By exploiting the microscopic silicon manufacturing variations of on-chip SRAM at power-up, the system generates a device-unique Physical Unclonable Function (PUF) fingerprint. Paired with a code-offset fuzzy extractor, the system dynamically reconstructs a stable 256-bit cryptographic key directly in volatile memory under μ T-Kernel 3.0 execution, eliminating the need to store persistent keys in non-volatile flash.

System Architecture & Memory Isolation

To prevent C-runtime initialization from wiping or contaminating the raw silicon state at boot:

- **Early Snapshot Pipeline:** An early routine (`puf_snapshot_early()`) captures uninitialized power-up states of SRAM2 and SRAM3 prior to calling `HAL_Init()` or system clock configuration.
- **Linker Partitioning:** Executable code, data sections, and `.bss` zero-initialization routines are explicitly restricted to SRAM1 via a custom linker script snippet (`linker_memory_snippet.ld`), leaving PUF target banks untouched.
- **Data Streaming:** Raw snapshots are streamed over the ST-LINK Virtual COM Port (VCP) to a Python host pipeline across multiple cold power-cycles for characterization.

Host Analysis & Fuzzy Extractor Pipeline

Due to environmental factors (thermal variation, voltage drift), raw SRAM reads contain noisy bit-flips. A host-side Python suite processes and stabilizes the fingerprint:

1. **Statistical Cell Selection:** Rather than using complex neural networks that risk leaking weights, `enroll.py` computes per-cell flip probabilities over N power cycles to select the 3,840 most reliable cells.
2. **Code-Offset Fuzzy Extractor:**
 - **Enrollment:** Computes non-secret helper data $H = R \oplus \text{repeat}(S)$, where R is the enrolled response and S is a 256-bit secret.
 - **Repetition Decoding ($n = 15$):** During boot, current response W evaluates $W \oplus H$. A majority vote across 15-cell blocks corrects up to 7 bit-errors per block.
 - **Key Derivation:** The decoded secret is hashed to yield $\text{Key} = \text{SHA256}(\text{tag} \parallel S)$ and validated against a pre-enrolled tag.

μ T-Kernel 3.0 Integration & Hardware Security

The key reconstruction workflow is encapsulated into a dedicated, high-priority task within μ T-Kernel 3.0:

- **Task Management:** The PUF task handles key extraction on system startup, notifies dependent security tasks via eventflags, and clears intermediate key buffers from memory once verified.
- **TrustZone Readiness:** Designed for optional ARM Cortex-M33 TrustZone partitioning, enabling the PUF extractor and key generation engine to execute within the Secure World while the main μ T-Kernel OS runs in Non-Secure memory.

Prototype Scope & System Constraints

- **Power-Cycle Dependency:** The system requires full physical cold power-cycles to regenerate the startup state, as warm system resets or debugger restarts preserve volatile SRAM contents.
- **Entropy Budget:** The repetition code implementation is designed as a single-board proof of concept; production deployments require expanding the entropy pool to account for helper-data leakage.
- **Environmental Scope:** Helper-data authentication is kept outside the initial prototype scope, focusing strictly on core PUF stabilization under environmental variations.